

# Memoization Example: Online Shopping Discount Calculation

---

Practical **Memoization** example using an **online shopping discount calculation**.

## Problem

We have a list of customers purchasing products.

Each record contains:

- Customer ID
- Product
- Original Price
- Discount %
- Card Name

The final amount depends **only on the product, price, discount %, and card type**.

If another customer has the **same combination**, we reuse the previously calculated amount instead of recalculating.

---

## Input

```
customers = [  
    ("C101", "Laptop", 60000, 10, "HDFC"),  
    ("C102", "Laptop", 60000, 10, "HDFC"),  
    ("C103", "Mobile", 30000, 5, "SBI"),  
    ("C104", "Laptop", 60000, 10, "HDFC"),  
    ("C105", "Mobile", 30000, 5, "SBI"),  
    ("C106", "Tablet", 20000, 8, "ICICI")  
]
```

---

## Python Program (Memoization)

```
# Memoization dictionary
cache = {}

# Function to calculate final amount
def calculate_price(product, price, discount, card):

    # Unique key
    key = (product, price, discount, card)

    # Check if already calculated
    if key in cache:
        print(f"Retrieved from Cache -> {product}")
        return cache[key]

    print(f"Calculating -> {product}")

    # Product discount
    amount = price - (price * discount / 100)

    # Card discount
    if card == "HDFC":
        amount -= 1000
    elif card == "SBI":
        amount -= 500
    elif card == "ICICI":
        amount -= 750

    cache[key] = amount
    return amount

customers = [
    ("C101", "Laptop", 60000, 10, "HDFC"),
    ("C102", "Laptop", 60000, 10, "HDFC"),
    ("C103", "Mobile", 30000, 5, "SBI"),
    ("C104", "Laptop", 60000, 10, "HDFC"),
    ("C105", "Mobile", 30000, 5, "SBI"),
    ("C106", "Tablet", 20000, 8, "ICICI")
]

print("Customer ID\tFinal Amount")

for cid, product, price, discount, card in customers:
```

```
final_amount = calculate_price(product, price, discount, card)
print(f"{cid}\t\t₹{final_amount:.2f}")
```

---

## Output

| Customer ID | Final Amount |
|-------------|--------------|
|-------------|--------------|

Calculating -> Laptop

|      |           |
|------|-----------|
| C101 | ₹53000.00 |
|------|-----------|

Retrieved from Cache -> Laptop

|      |           |
|------|-----------|
| C102 | ₹53000.00 |
|------|-----------|

Calculating -> Mobile

|      |           |
|------|-----------|
| C103 | ₹28000.00 |
|------|-----------|

Retrieved from Cache -> Laptop

|      |           |
|------|-----------|
| C104 | ₹53000.00 |
|------|-----------|

Retrieved from Cache -> Mobile

|      |           |
|------|-----------|
| C105 | ₹28000.00 |
|------|-----------|

Calculating -> Tablet

|      |           |
|------|-----------|
| C106 | ₹17650.00 |
|------|-----------|

---

## How Memoization Works

---

### Customer C101

Laptop

Price = ₹60,000

Discount = 10%

After discount

= 60000 - 6000

= ₹54,000

HDFC Card Discount = ₹1,000

Final Amount = ₹53,000

Store in cache

("Laptop", 60000, 10, "HDFC")

↓

₹53,000

---

## Customer C102

Same product, price, discount, and card.

Instead of calculating again,

Cache Lookup

("Laptop", 60000, 10, "HDFC")

↓

₹53,000

Value is returned immediately.

---

## Customer C103

Mobile

Price = ₹30,000

5% Discount

= ₹1,500

After discount

= ₹28,500

SBI Card Discount

= ₹500

Final Amount

= ₹28,000

Stored in cache

```
("Mobile", 30000, 5, "SBI")
```

↓

₹28,000

---

## Customer C105

Same details as Customer C103.

Retrieved from Cache

```
("Mobile", 30000, 5, "SBI")
```

↓

₹28,000

No recalculation is performed.

---

## Cache Contents After Execution

---

```
{  
  ('Laptop', 60000, 10, 'HDFC'): 53000,  
  ('Mobile', 30000, 5, 'SBI'): 28000,  
  ('Tablet', 20000, 8, 'ICICI'): 17650  
}
```

---

## Time Complexity

---

- **Without Memoization:**  $O(n \times \text{calculation\_cost})$  because every customer's discount is computed.
  - **With Memoization:**  $O(n)$  average, since repeated combinations are fetched from the cache in constant time ( $O(1)$  average lookup).
-

# Key Idea

---

This example clearly demonstrates the core idea of **Memoization**:

**Compute once, store the result, and reuse it whenever the same input appears again.**